

W07D06 — Code Review & Project Polish

ML Engineer Journey · Phase 2 — API & Deploy · Week 7, Day 6

1. Kenapa Hari Ini Penting

W07D06 bukan hari belajar topik baru — ini adalah hari **bersih-bersih** sebelum deploy. Engineer junior menulis kode yang jalan. Engineer yang layak menulis kode yang bisa dibaca dan di-maintain oleh orang lain — termasuk dirinya sendiri 6 bulan ke depan. Week 8 adalah deployment ke production. Kode yang messy di lokal akan jauh lebih messy di production.

Analogi: Bayangkan kamu baru selesai masak. Makanannya enak, bisa dimakan — tapi dapur kacau. Besok ada tamu penting yang mau ikut masak di dapur itu. Code review adalah membersihkan dapur sebelum tamu datang.

2. Apa Itu Kode 'Messy'?

Kode messy bukan kode yang tidak jalan. Kode messy adalah kode yang sulit dibaca, sulit diubah, dan berisiko menyebabkan bug saat di-maintain.

Jenis Masalah	Contoh Buruk	Contoh Benar
Magic number	if score > 0.73:	RISK_THRESHOLD = 0.73 if score > RISK_THRESHOLD:
Dead code	# def old_predict(): # return model...	Hapus seluruhnya
Fungsi terlalu panjang	def process(): # 80 baris	Pecah jadi 2-3 fungsi SRP
Import tidak dipakai	import json # tak pernah dipakai	Hapus import tersebut
Naming inkonsisten	def GetData() dan get_user()	Konsisten: snake_case semua

3. Workflow Code Review — Urutan yang Benar

Step 1 — Jalankan linter dulu

Ruff mendeteksi masalah sintaksis secara otomatis: unused imports, naming violations, undefined names. Selesaikan ini dulu sebelum baca manual.

```
pip install ruff ruff check . ruff check . --fix
```

Step 2 — Baca kode manual dari atas ke bawah

Ruff tidak bisa mendeteksi: magic numbers, fungsi yang terlalu panjang, logika yang bocor ke layer yang salah, atau naming yang ambigu tapi valid secara sintaksis. Ini butuh mata manusia.

Prioritas file: main.py → app/routers/ → app/services/ → app/schemas/

Step 3 — Cek secrets dan .env

```
git status # .env tidak boleh muncul cat .env.example # pastikan up-to-date
```

Penting: .env.example hanya cantumkan variabel yang benar-benar dipakai project saat ini. Jangan overpromise — variabel database yang belum dipakai tidak perlu ada.

Step 4 — Smoke test semua endpoint

Request valid → harus 200. Missing field → harus 422 (bukan 500). Tipe data salah → harus 422 dengan pesan yang jelas. Jika ada endpoint yang return 500 untuk input invalid, itu bug — perbaiki sebelum commit.

4. Ruff (Linter) vs Manual Review — Keduanya Wajib

Aspek	Ruff (Otomatis)	Manual Review
Kecepatan	Detik	Menit-jam
Apa yang dideteksi	Sintaksis, imports, naming PEP8	Logika, arsitektur, magic numbers
Bisa digantikan?	Tidak — terlalu lambat kalau manual	Tidak — ruff tidak paham konteks
Kapan dijalankan	Pertama kali, sebelum baca manual	Setelah ruff bersih

5. Conventional Commits — Format Standar Industri

Commit message bukan sekadar catatan untuk diri sendiri — ini adalah dokumentasi yang bisa dibaca 6 bulan ke depan tanpa harus membuka kode. Format standar:

```
( ): [body opsional – jelaskan WHY, bukan WHAT]
```

Type	Kapan Dipakai	Contoh
feat	Fitur baru	feat(predict): add confidence score to response
fix	Perbaikan bug	fix(auth): handle missing token returning 500 not 401
refactor	Ubah kode tanpa ubah behavior	refactor(service): extract threshold to named constant
docs	Perubahan dokumentasi	docs(readme): add architecture diagram section
chore	Maintenance, update deps	chore: clean up .env.example
style	Formatting, whitespace	style: fix trailing whitespace in routers

Aturan penting: satu commit = satu perubahan logis. Jangan campur refactor dan feature baru dalam satu commit — sulit di-revert kalau ada masalah.

6. Checklist Pre-Deploy Project 1

	Item	Keterangan
■	Linter	ruff check . mengembalikan 0 error
■	Magic numbers	Semua konstanta punya nama deskriptif (bukan 'variable = 0.035')
■	Dead code	Tidak ada kode yang dikomentari dan dibiarkan
■	Imports	Tidak ada import yang tidak dipakai
■	Naming	snake_case konsisten di seluruh codebase
■	.env	.env tidak ter-track git, .env.example ada dan akurat
■	Folder structure	routers/ schemas/ services/ main.py — setiap layer punya tanggung jawabnya
■	Error handling	Input invalid return 422, bukan 500

■	Smoke test	GET /health → 200, POST /predict valid → 200, POST /predict invalid → 422
■	Commit message	Menggunakan conventional commits dengan deskripsi yang jelas

7. Pelajaran Penting yang Harus Diingat

1. Kode yang jalan bukan berarti kode yang baik. Engineer yang layak menulis kode untuk manusia, bukan hanya untuk komputer.
2. Nama konstanta harus deskriptif: TRANSACTION_FEE_RATE = 0.035, bukan variable = 0.035. Nama yang buruk sama buruknya dengan magic number.
3. Ruff dan manual review tidak saling menggantikan — ruff untuk sintaksis, manual untuk konteks dan logika.
4. Conventional commits adalah dokumentasi, bukan formalitas. 'fix stuff' tidak membantu siapapun, termasuk dirimu sendiri.
5. .env.example harus jujur — hanya cantumkan variabel yang benar-benar dibutuhkan project saat ini.
6. 422 Unprocessable Entity adalah respons yang benar untuk input invalid. 500 Internal Server Error untuk input user adalah bug.
7. Code review bukan aktivitas sekali — ini habit yang harus dilakukan sebelum setiap commit penting.